

DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

- CLI To-Do List Manager - Technical Specification & Architecture Document 3**
 - 1. Executive Summary & Architecture Overview 3
 - 1.1 Executive Brief 3
 - 1.2 Maturity Assessment 3
 - 1.3 Technical Stack 3
 - 1.4 Architectural Constraints 3
 - 1.5 Critical Dependencies 3
 - 2. Architecture Workflows 4
 - 2.1 Project Implementation Roadmap & Traceability 7
 - 2.2 CLI Command Execution Workflow 7
 - 2.3 Task Management Sequence 7
 - 2.4 Data Model Entity Relationship 8
 - 3. Detailed Technical Specifications & Business Rules 9
 - 3.1 Requirements Traceability 10
 - 3.2 Security Rules 12
 - 3.3 Data Models 12
 - 4. Project Governance & Structural Gaps 13
 - 4.1 Structural Gaps 13
 - 4.2 Remediation & Workflow 13
 - 5. Technical & Domain Glossary (Terminology Reference) 13

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

This project implements a Python-based CLI To-Do List Manager utilizing a local JSON file for persistence. The architecture is decomposed into a shared service layer and a command-line interface, following a phased implementation strategy where foundational infrastructure blocks all user-story-specific development. The system focuses on core task lifecycle management: creation, sequential ID assignment, listing (human-readable and JSON), and removal.

1.2 Maturity Assessment

The specification is logically structured with a clear dependency graph, but it is currently in REFINEMENT. While the implementation roadmap is comprehensive, there are high-severity structural gaps regarding formal Acceptance Criteria and a complete absence of security constraints (e.g., input validation and shell injection prevention) and performance bounds for the JSON storage mechanism.

1.3 Technical Stack

- Python
- pyproject.toml

1.4 Architectural Constraints

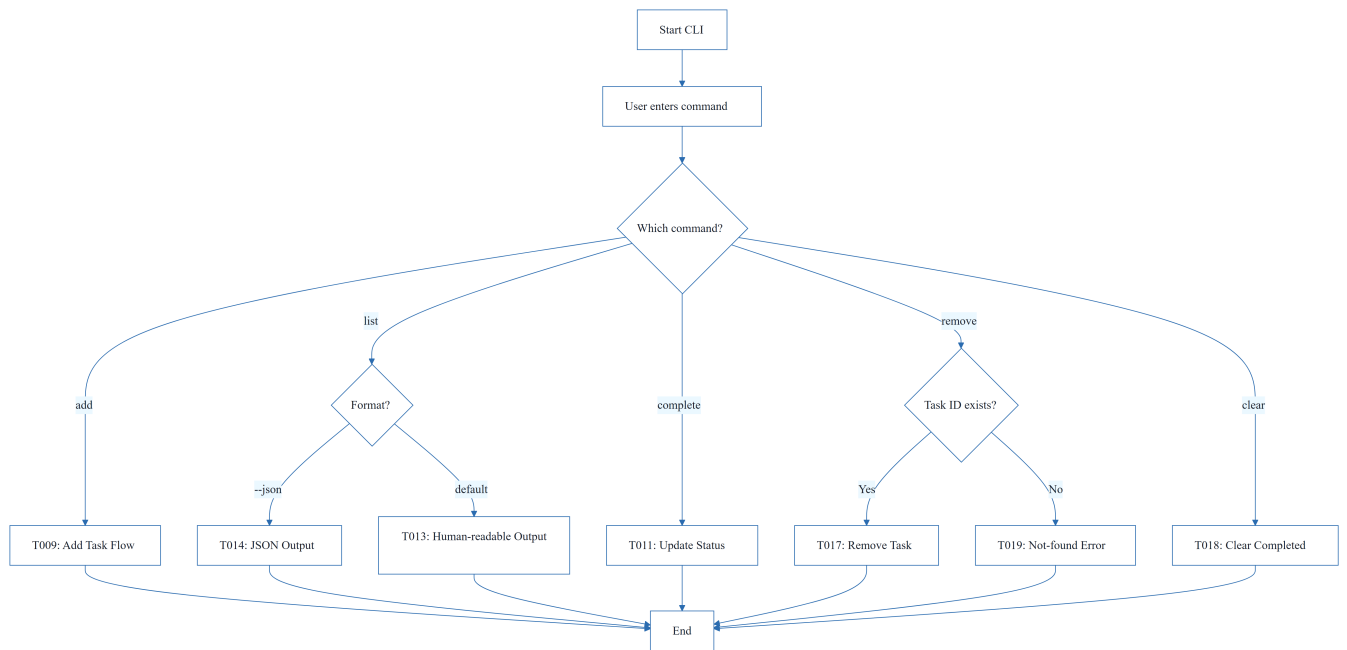
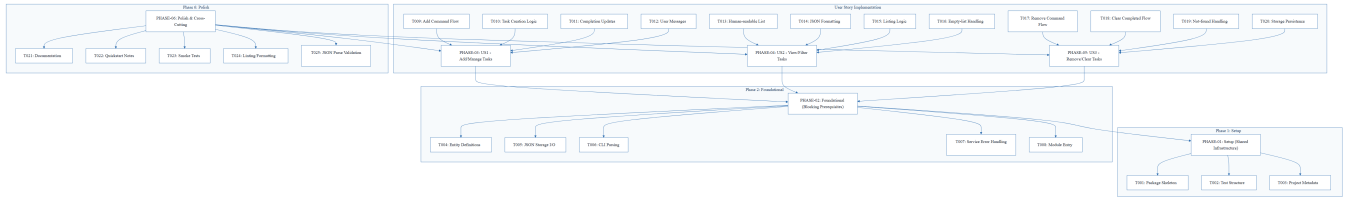
- Sequential ID assignment for task entities.
- JSON serialization for data persistence.
- Strict sequential phase execution: Phase 1 (Setup) -> Phase 2 (Foundational) -> User Stories.
- Persistence target: `~/ . todos . json`.
- Parseable JSON output requirement for `todo list --json` command.
- Module entry behavior restricted to `python -m todo_manager`.

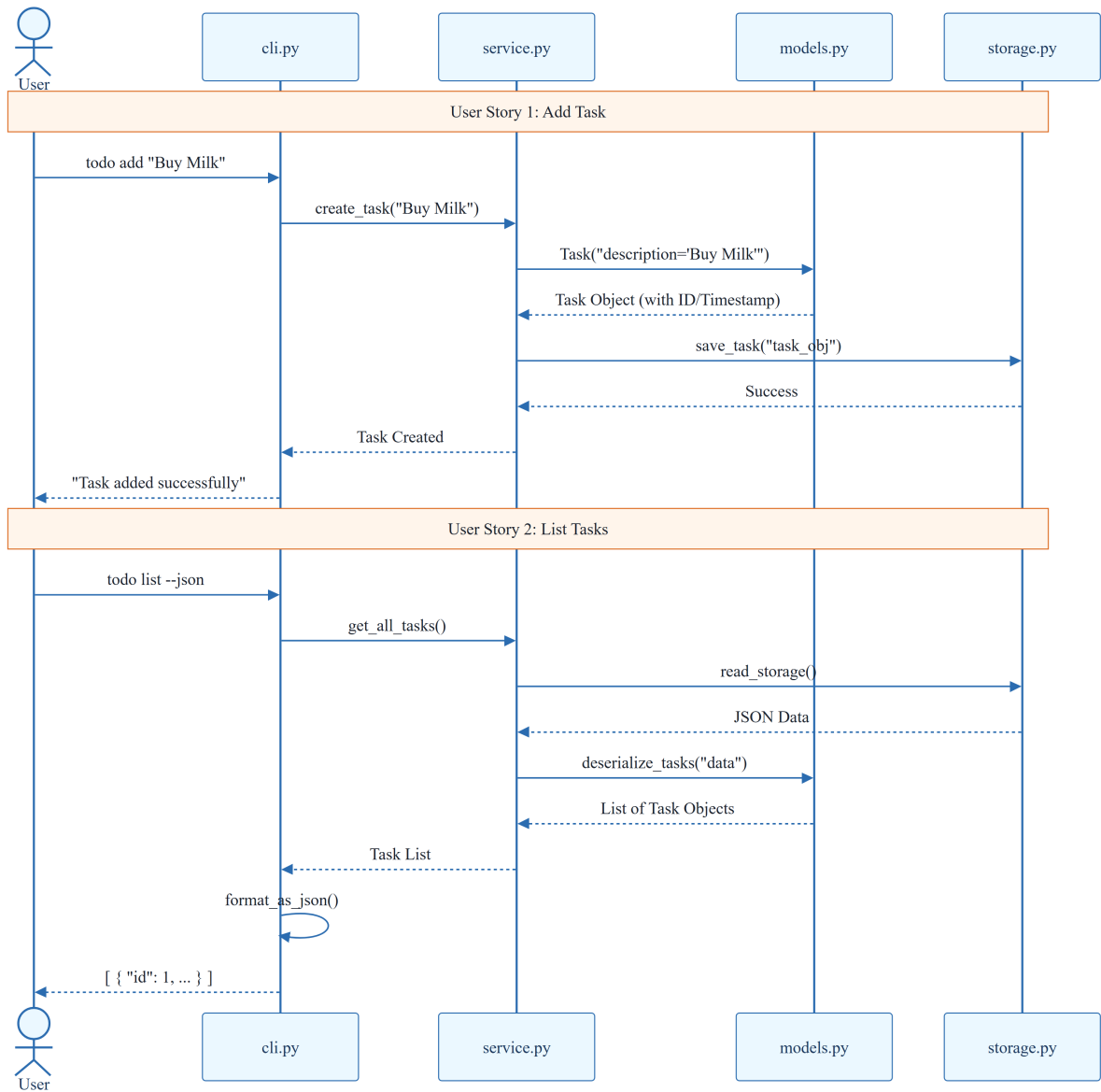
1.5 Critical Dependencies

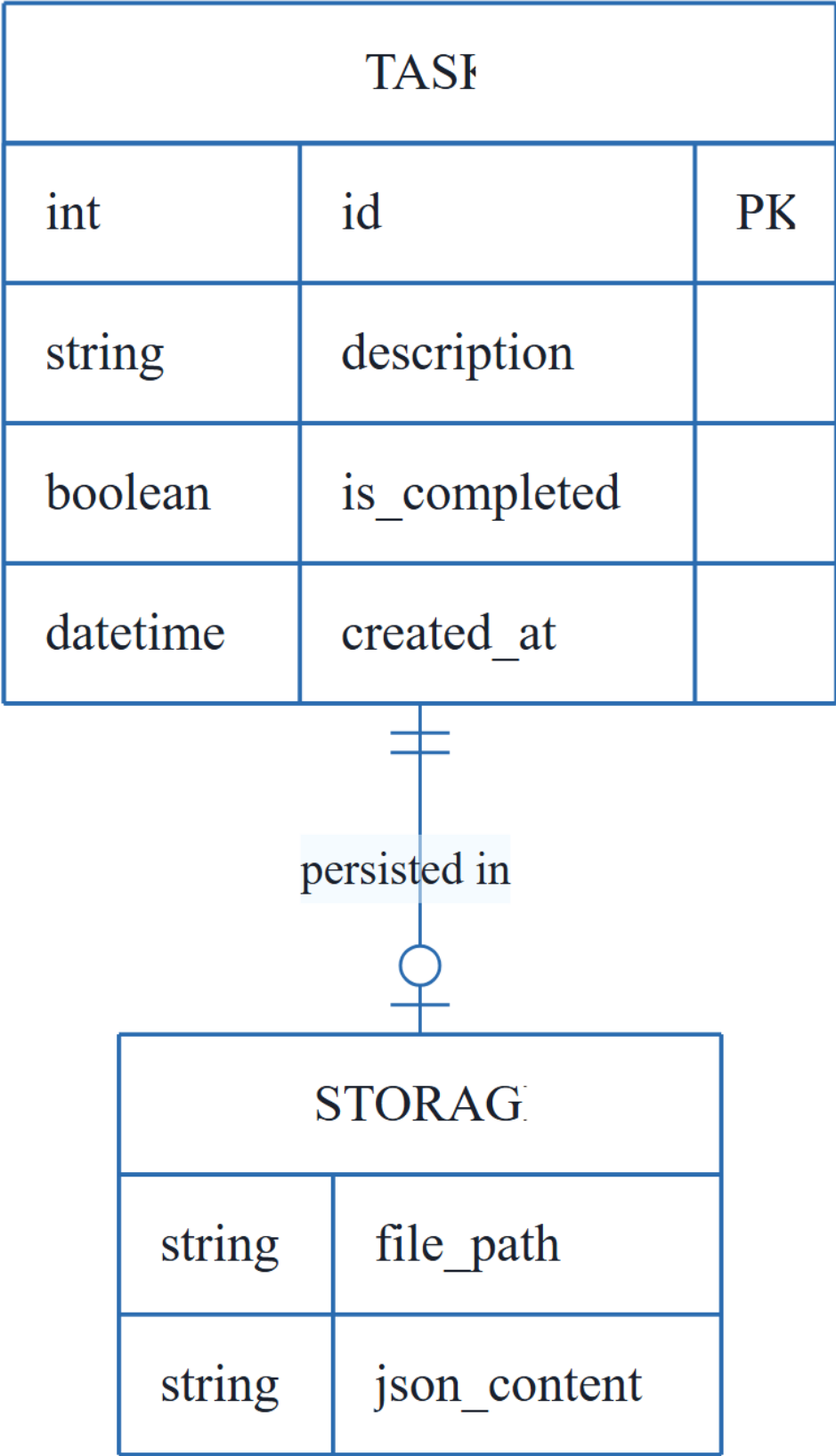
- Foundational Phase (PHASE-02) completion is a blocking prerequisite for all User Story implementation.
- JSON file I/O helpers in `src/todo_manager/storage.py` for state persistence.
- Referential integrity between task IDs and the JSON storage backend.

- `pyproject.toml` configuration for CLI entry points and tooling.

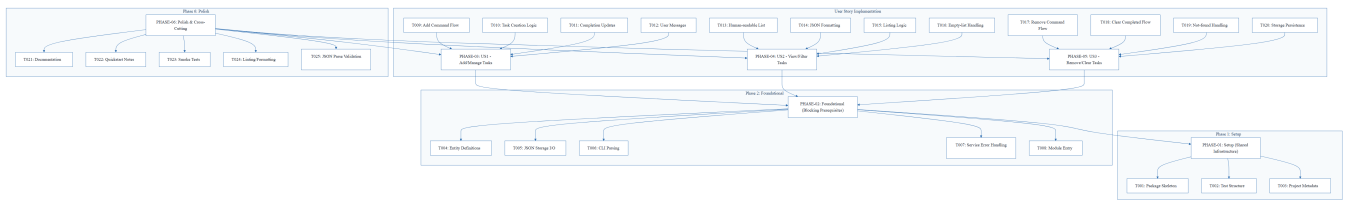
2. Architecture Workflows



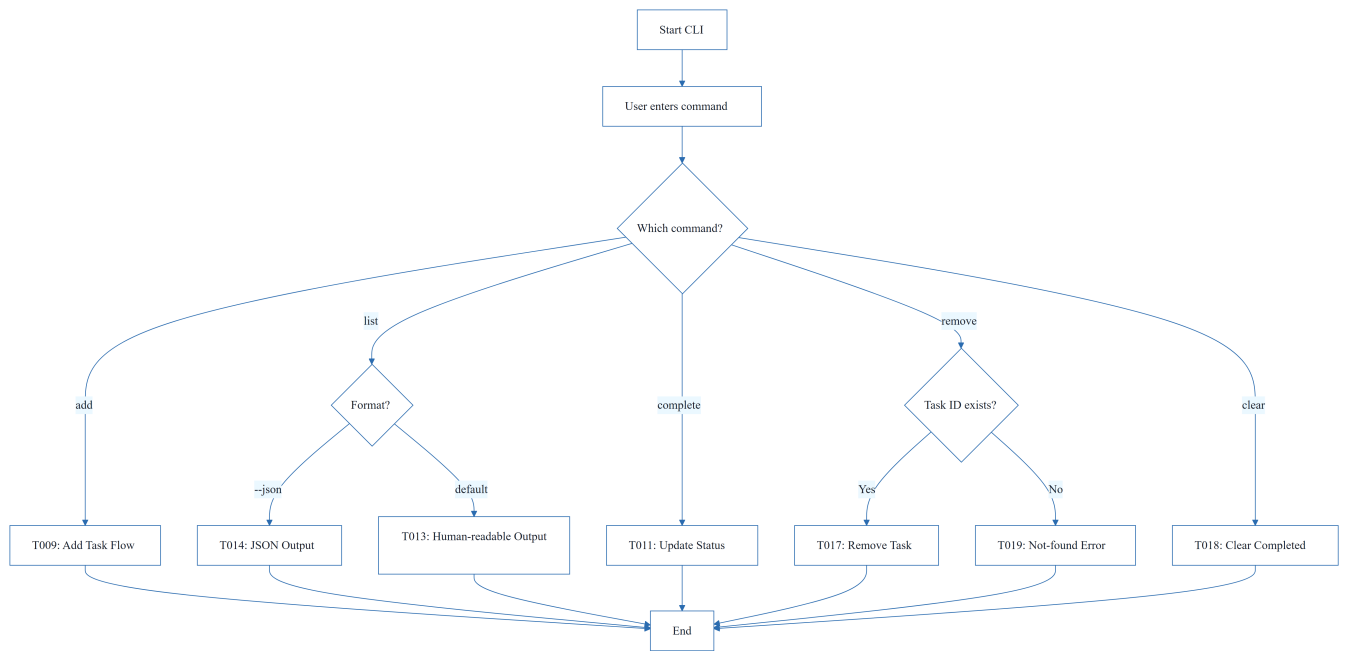




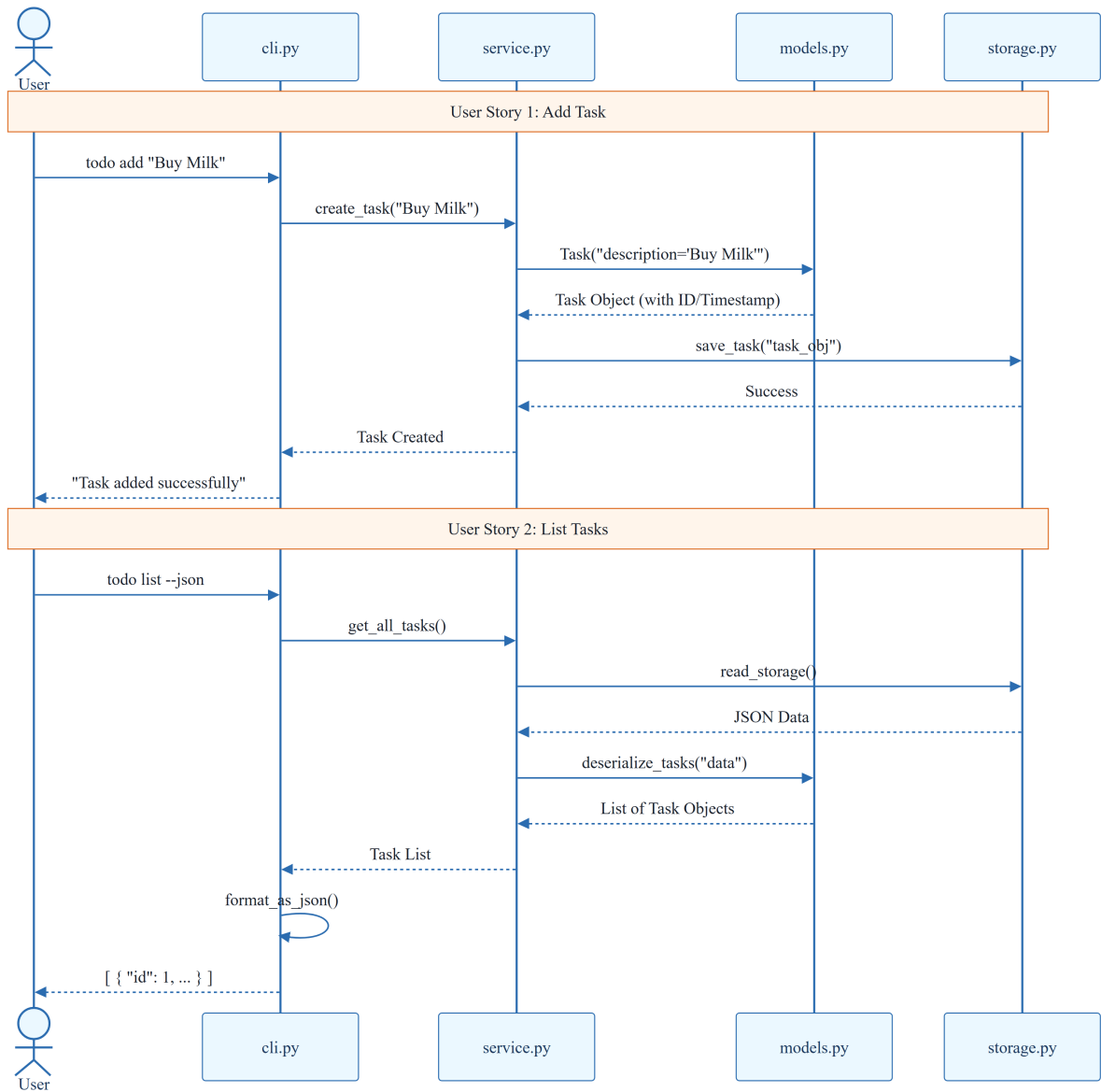
2.1 Project Implementation Roadmap & Traceability



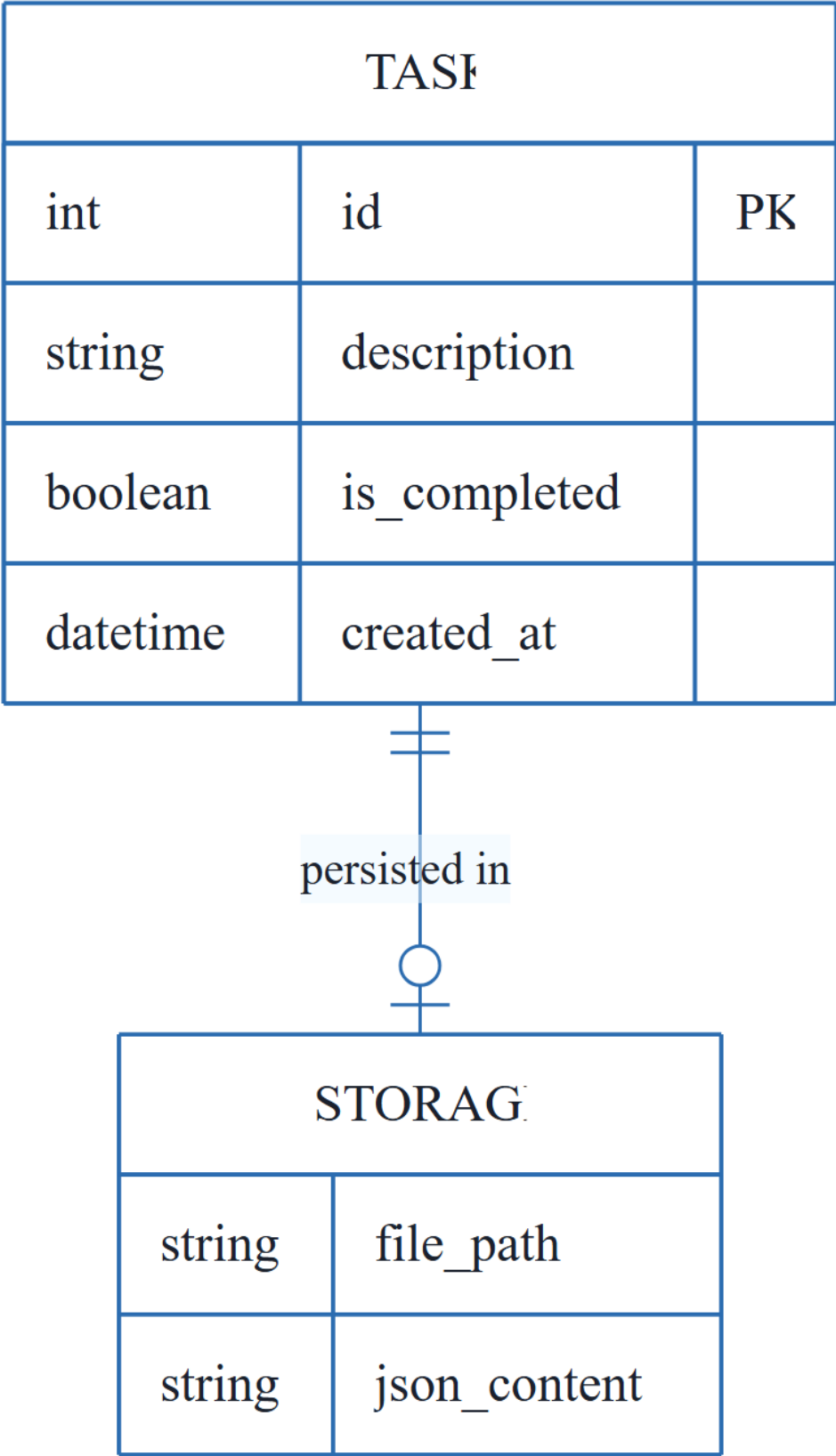
2.2 CLI Command Execution Workflow



2.3 Task Management Sequence



2.4 Data Model Entity Relationship



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

ID	Requirement / Task Description	Source Section	Priority / Story
T001	Create the Python package skeleton in <code>src/todo_manager/init.py</code> , <code>main.py</code> , <code>cli.py</code> , <code>models.py</code> , <code>storage.py</code> , and <code>service.py</code>	Phase 1: Setup	N/A
T002	Create the test directory structure in <code>tests/unit/</code> and <code>tests/integration/</code>	Phase 1: Setup	N/A
T003	Add project metadata and tooling entry points in <code>pyproject.toml</code>	Phase 1: Setup	N/A
T004	Implement task entity definitions and serialization helpers in <code>src/todo_manager/models.py</code>	Phase 2: Foundational	N/A
T005	Implement JSON storage path resolution and file I/O helpers in <code>src/todo_manager/storage.py</code>	Phase 2: Foundational	N/A
T006	Implement shared command-line parsing and top-level dispatch in <code>src/todo_manager/cli.py</code>	Phase 2: Foundational	N/A
T007	Implement shared service-layer error handling and task collection utilities in <code>src/todo_manager/service.py</code>	Phase 2: Foundational	N/A
T008	Define module entry behavior for python <code>-m todomanager</code> in <code>src/todomanager/main.py</code>	Phase 2: Foundational	N/A
T009	Implement the add command flow in <code>src/todomanager/cli.py</code> and <code>src/todomanager/service.py</code>	Implementation for US1	US1
T010	Implement task creation, sequential ID assignment, and <code>created_at</code> population in <code>src/todo_manager/models.py</code>	Implementation for US1	US1

ID	Requirement / Task Description	Source Section	Priority / Story
T011	Implement completion updates for existing tasks in <code>src/todo_manager/service.py</code>	Implementation for US1	US1
T012	Add user-facing success and error messages for add and complete operations in <code>src/todo_manager/cli.py</code>	Implementation for US1	US1
T013	Implement human-readable task listing output in <code>src/todo_manager/cli.py</code>	Implementation for US2	US2
T014	Implement <code>--json</code> output formatting in <code>src/todo_manager/cli.py</code>	Implementation for US2	US2
T015	Add listing logic that preserves task order and status fields in <code>src/todo_manager/service.py</code>	Implementation for US2	US2
T016	Handle the empty-list case with a clear message in <code>src/todo_manager/cli.py</code>	Implementation for US2	US2
T017	Implement the <code>remove</code> command flow in <code>src/todomanager/cli.py</code> and <code>src/todomanager/service.py</code>	Implementation for US3	US3
T018	Implement the <code>clear</code> command flow for removing completed tasks in <code>src/todo_manager/service.py</code>	Implementation for US3	US3
T019	Add not-found handling for invalid task IDs in <code>src/todo_manager/cli.py</code>	Implementation for US3	US3
T020	Ensure storage writes persist removals and clears safely in <code>src/todo_manager/storage.py</code>	Implementation for US3	US3
T021	Document the CLI usage and storage behavior in <code>README.md</code>	Phase 6: Polish	N/A
T022	Add quickstart verification notes and examples in <code>specs/001-cli-todo-manager/quickstart.md</code>	Phase 6: Polish	N/A

ID	Requirement / Task Description	Source Section	Priority / Story
T023	Run a manual smoke test of add, list, complete, remove, and clear against ~/.todos.json	Phase 6: Polish	N/A
T024	Verify linting and formatting expectations for the source files in src/todo_manager/	Phase 6: Polish	N/A
T025	Confirm the generated JSON output remains parseable for the todo list --json path	Phase 6: Polish	N/A
TEST-US1	Run <code>todo add "Task description"</code> and <code>todo complete</code> , then confirm stored task list reflects new task and completion status.	Phase 3: US1	US1
TEST-US2	Run <code>todo list</code> and <code>todo list --json</code> and confirm output is readable or parseable JSON.	Phase 4: US2	US2
TEST-US3	Run <code>todo remove</code> and <code>todo clear</code> , then confirm target/completed tasks are removed.	Phase 5: US3	US3

3.2 Security Rules

- **Input Validation:** No explicit security constraints defined. Remediation required to prevent shell injection in CLI arguments.
- **Data Integrity:** Sequential ID assignment must be maintained to prevent collisions.

3.3 Data Models

- **Task Entity:**
 - `id` (int): Primary Key, sequential.
 - `description` (string): Text content of the task.
 - `is_completed` (boolean): Completion status.
 - `created_at` (datetime): Timestamp of creation.
- **Storage Model:**
 - `file_path` (string): Path to `~/.todos.json`.
 - `json_content` (string): Serialized JSON array of Task entities.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap	Priority	Remediation Advice
Acceptance Criteria	HIGH	The document defines 'Goals' and 'Tests' for each story, but lacks a formal list of acceptance criteria (e.g., 'Should handle special characters in task descriptions').
Security & Performance Constraints	MEDIUM	No constraints mentioned regarding the size of the JSON file or input validation security (e.g., preventing shell injection in CLI arguments).
Open Questions & Uncertainties	LOW	Declaring any remaining unknowns regarding the OS file system behavior for <code>~/ .todos.json</code> would be beneficial.

4.2 Remediation & Workflow

1. **Immediate:** Define formal Acceptance Criteria for US1, US2, and US3.
2. **Short-term:** Implement input sanitization for CLI arguments to mitigate injection risks.
3. **Mid-term:** Define performance bounds for the JSON storage (e.g., maximum number of tasks before performance degrades).

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
Checkpoint	TECHNICAL_STACK	PHASE-02	A synchronization milestone verifying that base infrastructure is operational before initiating parallel feature development.
Foundational	TECHNICAL_STACK	PHASE-02	The blocking layer of shared infrastructure, including serialization and I/O helpers, upon which all subsequent features depend.
Goal	BUSINESS_DOMAIN	PHASE-03	The high-level functional objective that a specific user story must achieve to satisfy the end-user requirement.

Term	Category	Context Anchor	Project Definition
ID	BUSINESS_DOMAIN	T010	A unique sequential numeric identifier assigned to each task entity during its creation process.
JSON	TECHNICAL_STACK	T005	The lightweight data-interchange format used for persistent storage in <code>~/todos.json</code> and for specific command-line output flags.
MVP	BUSINESS_DOMAIN	PHASE-03	The minimal set of functional capabilities, centered on creating and completing tasks, required for the first viable release.
Organization	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The grouping strategy where implementation steps are clustered by user story to ensure independent testability.
Prerequisites	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The set of design documents, including data models and command contracts, that must be available before starting implementation.
README	TECHNICAL_STACK	T021	The primary documentation file detailing command-line interface usage and the behavior of the underlying storage.
Setup	TECHNICAL_STACK	PHASE-01	The initial phase consisting of package skeleton creation, directory structuring, and <code>pyproject.toml</code> configuration.
T016	TECHNICAL_STACK	T016	The specific implementation step for handling and messaging the state where the task collection is empty.
Tests	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	Verification procedures executed via quickstart checks and smoke tests to validate operational correctness.
population in	BUSINESS_DOMAIN	T010	The process of assigning a timestamp value to the <code>created_at</code> attribute during the instantiation of a task.

Term	Category	Context Anchor	Project Definition
todo clear	BUSINESS_DOMAIN	T018	The operational command intended to purge all task entities that have been marked as completed from the persistent store.
todo list	BUSINESS_DOMAIN	T013	The operational command for retrieving and displaying all stored tasks in either a human-readable or machine-parseable format.
■■ CRITICAL	TECHNICAL_STACK	PHASE-02	A high-severity dependency marker indicating that subsequent work is strictly blocked until the associated phase is finalized.